

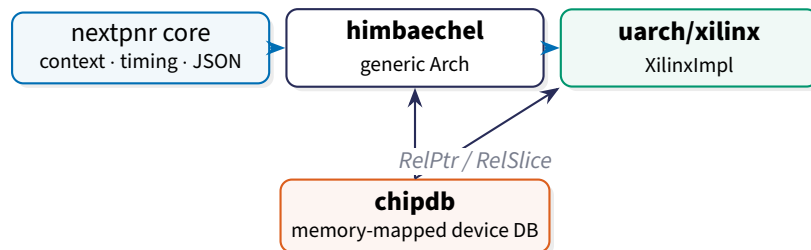
The Himbaechel Architecture

How nextpnr scales place-and-route to Xilinx 7-series

An illustrated guide to the chip database, the uarch plugin API, and the
xc7 implementation in `himbaechel/uarch/xilinx`

nextpnr documentation working group

August 19, 2026 · nextpnr `nextpnr-0.11.1` (62e659e)



About this document

This guide explains *how himbaechel works* from the inside: the deduplicated chip database and its relative-pointer model, the uarch plugin interface, and how the Xilinx 7-series uarch implements packing, placement, routing, timing, and FASM bitstream generation on top of it. It is written for engineers who already know what place-and-route does and want to understand this specific implementation well enough to modify it. Code excerpts are taken verbatim from the source tree at tag `nextpnr-0.11.1`; line numbers refer to that checkout.

Contents

1	Motivation: why himbaechel exists	2
2	Architecture at a glance	2
3	The chip database	3
3.1	Generation pipeline	3
3.2	Relative pointers: RelPtr and RelSlice	3
3.3	Loading and validating	4
4	The device model: tiles, wires, pips, nodes	5
4.1	Tile grid	5
4.2	Wire-to-node mapping	5
5	The uarch plugin API	6
6	The Xilinx uarch: packing	8
7	Placement: shared algorithms, family legality	9
7.1	Dispatch	9
7.2	HeAP: the analytical placer	10
7.3	What the Xilinx uarch configures	10
7.4	Legality: the heart of xc7 placement	11
8	Routing: router2 and the Xilinx glue	12
8.1	Delay estimation	13
9	Timing	13
10	FASM: from routed design to bitstream	13
11	Device support today	14
A	Glossary	14
B	Where the details live	15

1 Motivation: why himbaechel exists

nextpnr has two generations of architecture interfaces. The original one—colloquially “Viaduct”—builds a *flat, in-memory routing graph* at startup: every wire, pip, and bel is instantiated as a C++ object and linked together. That is fine for small devices (iCE40, ECP5, Gowin) but the object overhead explodes past roughly 20,000 logic elements. A Xilinx Artix-7 die has tens of thousands of slices, each with many pips and wires; a flat model would cost multiple gigabytes of RAM.

Himbaechel (the nextpnr maintainers’ name for the “bigger arches” framework, documented upstream in `docs/himbaechel.md`) solves this by moving the device model out of C++ objects and into a *compile-time generated, deduplicated binary database*—the *chipdb*—which is memory-mapped at runtime and read through *relative pointers*. Million-LUT devices then fit in roughly 100 MB instead of multi-GB, and most of the routing-graph queries become flat array lookups into POD (plain-old-data) structures.

Scaling in one sentence

Viaduct stores the device as a heap of interlinked C++ objects; himbaechel stores it as a position-independent binary blob of deduplicated PODs that is mapped into memory and traversed with relative pointers. The place-and-route *algorithms* stay shared; only the device model changes.

The framework supports multiple device families by delegating to per-family “micro-architectures” (uarchs): example, `gowin`, `gatemate`, `ng-ultra`, and—the subject of this guide—`xilinx` for Xilinx 7-series (xc7).

2 Architecture at a glance

compile time

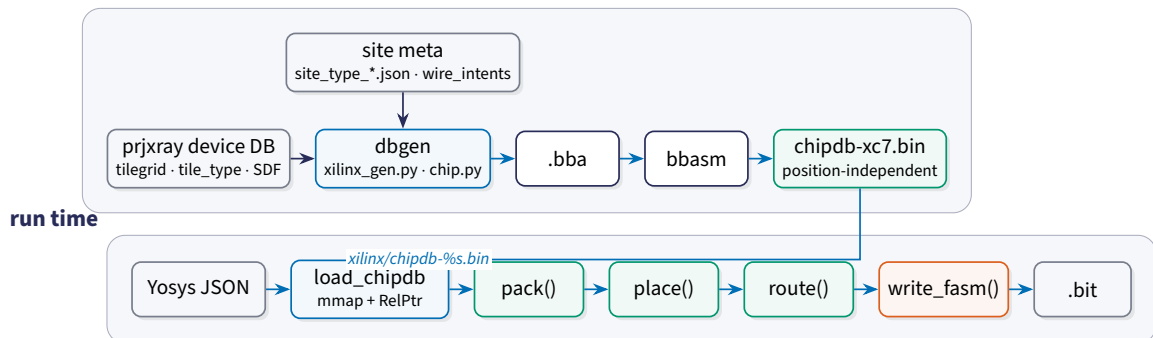


Figure 1: End-to-end data flow: a Python generator converts prjxray device data plus site metadata into a binary chipdb at build time; at run time nextpnr memory-maps the blob and runs the standard pack → place → route → FASM pipeline against it.

Figure 1 shows the whole system. The essential idea: *everything device-specific that can be precomputed is precomputed offline and shipped as one immutable binary file*. The runtime code queries that file; it never builds a routing graph object model.

Three code layers cooperate at run time:

- **The generic Arch** (`himbaechel/arch.h`, `arch.cc`)— a concrete subclass of the shared `BaseArch<ArchRanges>` that implements every nextpnr architecture API (bel/wire/pip iteration, names, bounds, delays, drawing) as queries over the chipdb PODs.

- **The uarch plugin** (one HimbaechelAPI subclass per family; for xc7: `XilinxImpl` in `uarch/xilinx/`)—supplies all the *family-specific knowledge*: packing, legality, placement configuration, clock routing, FASM emission.
- **The shared algorithms** (common/`place`, common/`route`)—the analytical placer (HeAP), the ePlace-style static placer, the SA placer, router1 and router2—which are family-agnostic and talk to the uarch only through callbacks.

3 The chip database

3.1 Generation pipeline

The chipdb is produced at *build time*. The Xilinx generator lives in `himbaechel/uarch/xilinx/gen/` and is driven by CMake (`add_bba_produce_command + add_bba_compile_command`):

`himbaechel/uarch/xilinx/CMakeLists.txt:32`

```
1 set(ALL_HIMBAECHEL_XILINX_DEVICES xc7a100t xc7a200t xc7a50t xc7s50 xc7z010 xc7z020)
2 set(HIMBAECHEL_XILINX_DEVICES ${ALL_HIMBAECHEL_XILINX_DEVICES} CACHE STRING
3   "Include support for these Xilinx devices (available:
4     ${ALL_HIMBAECHEL_XILINX_DEVICES})")
5 if (HIMBAECHEL_XILINX_DEVICES STREQUAL "all")
6   set(HIMBAECHEL_XILINX_DEVICES ${ALL_HIMBAECHEL_XILINX_DEVICES})
7 endif()
8 message(STATUS "Enabled Himbaechel-Xilinx devices: ${HIMBAECHEL_XILINX_DEVICES}")
```

For each enabled device, `xilinx_gen.py` imports the prjxray database (`tilegrid.json`, `tile_type_*.json`, `package_pins.csv`, `tileconn.json`, SDF timings) plus the site meta-data from the `nextpnr-xilinx-meta` submodule, and emits a text `.bba` file. `bbasm` (the BBA assembler) compiles that into the final position-independent binary blob. The key generation steps:

1. **Enumerate tile types** and deduplicate them—a design with hundreds of identical CLB tiles stores *one* `TileTypePOD`, not hundreds.
2. **Build the wire/node graph**: intra-tile wires stay local; cross-tile connectivity is expressed through *nodes* that merge electrically-connected wires (see §4).
3. **Record sites and bels** from prjxray site types, filtered and remapped by `filters.py` (e.g. every 6LUT becomes a `SLICE_LUTX bel`).
4. **Import timing**: pip and node delays from prjxray `src_to_dst` data, cell arcs from SDF (`parse_sdf.py`).
5. **Serialize** everything through relative pointers (`himbaechel_dbgen/chip.py`) into `.bba`, then assemble to binary.

3.2 Relative pointers: `RelPtr` and `RelSlice`

The blob is position-independent because all cross-references inside it are *relative*: a `RelPtr<T>` stores a signed offset from its own address to the target, and `RelSlice<T>` stores an offset plus a count. Dereferencing adds the offset to the pointer's own location—so the whole file can be mapped at any address and still be internally consistent.

The root structure is cast directly from the mapped memory:

`himbaechel/chipdb.h:226`

```
1 NPNR_PACKED_STRUCT(struct ChipInfoPOD {
2   int32_t magic;
3   int32_t version;
4   int32_t width, height;
```

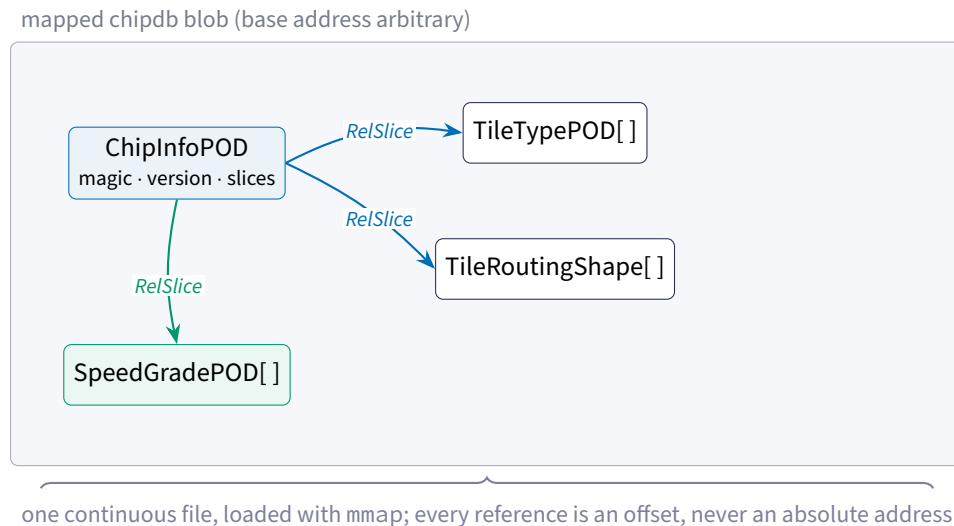


Figure 2: The chipdb is a single flat blob of POD arrays. The root `ChipInfoPOD` is found at the file start; every other structure is reached by relative offset from within the blob.

```

5
6     RelPtr<char> uarch;
7     RelPtr<char> name;
8     RelPtr<char> generator;
9
10    RelSlice<TileTypePOD> tile_types;
11    RelSlice<TileInstPOD> tile_insts;
12    RelSlice<NodeShapePOD> node_shapes;
13    RelSlice<TileRoutingShapePOD> tile_shapes;
14
15    RelSlice<PackageInfoPOD> packages;
16    RelSlice<SpeedGradePOD> speed_grades;
17
18    RelPtr<ConstIDDDataPOD> extra_constids;
19
20    RelPtr<uint8_t> extra_data;
21 });

```

3.3 Loading and validating

At startup the uarch asks the Arch to map its database:

```

himbaechel/arch.cc:117
1 void Arch::load_chipdb(const std::string &path)
2 {
3     std::string db_path;
4     if (!args.chipdb_override.empty()) {
5         db_path = args.chipdb_override;
6     } else {
7         db_path = proc_share_dirname();
8         db_path += "himbaechel/";
9         db_path += path;
10        std::filesystem::path p(db_path);
11        db_path = p.make_preferred().string();
12    }
13    try {
14        blob_file.open(db_path);
15        if (db_path.empty() || !blob_file.is_open())

```

```

16         log_error("Unable to read chipdb %s\n", db_path.c_str());
17         const char *blob = reinterpret_cast<const char *>(blob_file.data());
18         chip_info = get_chip_info(reinterpret_cast<const RelPtr<ChipInfoPOD> *>(blob));
19     } catch (...) {
20         log_error("Unable to read chipdb %s\n", db_path.c_str());
21     }
22     // Check consistency of blob
23     if (chip_info->magic != 0x00ca7ca7)
24         log_error("chipdb %s does not look like a valid himbaechel database!\n",
25             db_path.c_str());
26     if (chip_info->version != database_version)
27         log_error(
28             "chipdb uses db version %d but nextpnr is expecting version %d (did you
29             forget a database rebuild?).\n",
30             chip_info->version, database_version);
31     std::string blob_uarch(chip_info->uarch.get());
32     if (blob_uarch != args.uarch)
33         log_error("database device uarch '%s' does not match selected device uarch
34             '%s'.\n", blob_uarch.c_str(),
35             args.uarch.c_str());
36     // Setup constids from database
37     for (int i = 0; i < chip_info->extra_constids->bba_ids.ssize(); i++) {
38         IdString::initialize_add(this, chip_info->extra_constids->bba_ids[i].get(),
39             i + chip_info->extra_constids->known_id_count);
40     }
41 }

```

Three consistency checks protect against mismatched binaries: the *magic number* 0x00ca7ca7 (“cat cat”, a himbaechel signature), the *database version* (6 in this checkout), and the *uarch name* stored in the blob. Extra constids (identifier strings) that exist only in the device database are registered into the IdString pool at the end—so the chipdb is also the source of device-specific identifiers.

The default search path is <share>/himbäechel/<path>; the Xilinx uarch requests xilinx/chipdb-%s.bin with the die name (xilinx.cc:88), and xc7a35t is aliased to the xc7a50t die (xilinx.cc:86–87).

4 The device model: tiles, wires, pips, nodes

4.1 Tile grid

The device is a rectangular grid of *tile instances*. Each instance references a deduplicated TileTypePOD (which holds the bels, wires, pips and groups common to all tiles of that type) and a TileRoutingShapePOD (which maps tile wires onto the global *node* graph). Tile names are synthesised as <prefix>X<X>Y<Y> (arch.cc:188–200).

4.2 Wire-to-node mapping

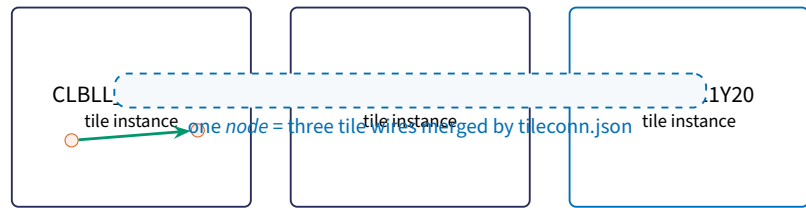
Each tile wire carries a RelNodeRefPOD that classifies it:

himbäechel/chipdb.h:116

```

1 NPNR_PACKED_STRUCT(struct RelNodeRefPOD {
2     // wire is entirely internal to a single tile
3     static constexpr int16_t MODE_TILE_WIRE = 0x7000;
4     // where this is the root {wire, dy} form the node shape index
5     static constexpr int16_t MODE_IS_ROOT = 0x7001;
6     // special cases for the global constant nets
7     static constexpr int16_t MODE_ROW_CONST = 0x7002;
8     static constexpr int16_t MODE_GLB_CONST = 0x7003;
9     // special cases where the user needs to outsmart the deduplication [0x7010, 0x7FFF]

```



node roots live in `node_shapes / wire_to_node`; a wire maps either to itself, to a node root, or to a row/global constant

Figure 3: Wires are local to tiles; electrical connectivity across tile boundaries is represented by *nodes*. The router moves between tile wires; `wire_to_node` tells it which wires are the same net.

```
10  static constexpr int16_t MODE_USR_BEGIN = 0x7010;
11  int16_t dx_mode; // relative X-coord, or a special value
12  int16_t dy;      // normally, relative Y-coord
13  uint16_t wire;   // normally, node index in tile (x+dx, y+dy)
14  });
```

- `MODE_TILE_WIRE`—the wire is its own node (no merging);
- `MODE_IS_ROOT`—this wire is the root of a merged node (carries the timing/delay info);
- `MODE_ROW_CONST / MODE_GLB_CONST`—the wire is part of a row- or globally-shared constant net (VCC/GND backbones).

Pips connect wires *within* a tile (or via the site pips inside a bel’s site); they carry a timing index into the speed-grade data and an opaque `extra_data` pointer used by the family (the Xilinx uarch stores `XlnxPipExtraDataPOD` there—e.g. LUT-permutation pip configuration):

`himbaechel/chipdb.h:74`

```
1  NPNR_PACKED_STRUCT(struct PipDataPOD {
2      int32_t src_wire;
3      int32_t dst_wire;
4
5      uint32_t type;
6      uint32_t flags;
7      int32_t timing_idx;
8
9      RelPtr<uint8_t> extra_data;
10 });
```

5 The uarch plugin API

The Himbaechel API interface is the contract every uarch must fulfil. The important hook groups:

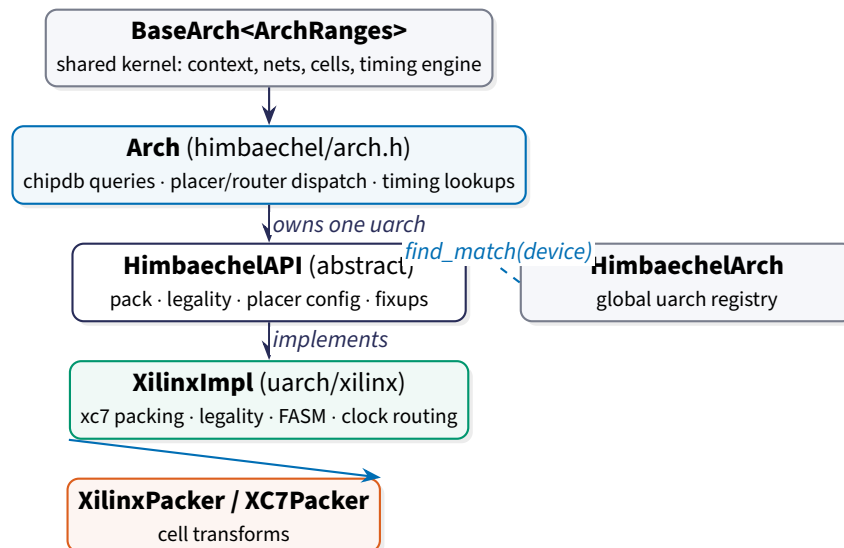


Figure 4: Class structure: the generic Arch wraps exactly one uarch plugin. At startup Arch::Arch() asks the global HimbaechelArch registry which uarch matches the requested device (arch.cc:43–58); for xc7* that is XilinxImpl.

Hook	Responsibility
<code>init_database()</code>	validate/configure the chipdb, set speed grade, seed tags
<code>getUArchOptions()</code>	register family CLI options (e.g. <code>-fasm</code> , <code>-xdc</code>)
<code>pack()</code>	transform the logical netlist into physical cells (bels)
<code>prePlace / postPlace</code>	tags, control-set indexing / post-placement fixups
<code>configurePlacerHeap / configurePlacerStatic</code>	feed family parameters into the shared placers
<code>checkBelAvail / checkPipAvail</code>	hard block resources (e.g. reserved pips)
<code>isBelLocationValid</code>	<i>legality</i> : can this set of cells share this site?
<code>preRoute / postRoute</code>	source/sink setup, clock routing / routing fixups, FASM
<code>getCellDelay / lookupCellSeqTimings</code>	timing arcs for the shared timing engine

The Xilinx uarch exposes exactly two CLI options of its own:

`himbaechel/uarch/xilinx/xilinx.cc:66`

```

1 XilinxImpl::~XilinxImpl() {};
2
3 po::options_description XilinxImpl::getUArchOptions()
4 {
5     po::options_description specific("Xilinx specific options");
6     specific.add_options()( "fasm", po::value<std::string>(), "fasm bitstream output file");
7     specific.add_options()( "xdc", po::value<std::string>(), "name of constraints file");
8     return specific;
9 }
  
```

6 The Xilinx uarch: packing

Packing rewrites the technology-mapped netlist (LUTs, FFs, CARRY4s, BRAMs, IOBs, ...) into *physical* cells that correspond 1:1 to chipdb bels. The xc7 implementation is spread over `himbaechel/uarch/xilinx/pack*.cc`; the top-level order is:

`himbaechel/uarch/xilinx/pack.cc:821`

```
1 void XilinxImpl::pack()
2 {
3     const ArchArgs &args = ctx->args;
4     if (args.options.count("xdc")) {
5         parse_xdc(args.options["xdc"].as<std::string>());
6     }
7
8     XC7Packer packer(ctx, this);
9     packer.pack_constants();
10    packer.pack_inverters();
11    packer.pack_io();
12    packer.prepare_clocking();
13    packer.pack_constants();
14    packer.pack_iologic();
15    packer.pack_idelayctrl();
16    packer.pack_clocking();
17    packer.generate_constraints();
18    packer.pack_muxfs();
19    packer.pack_carries();
20    packer.pack_srls();
21    packer.pack_luts();
22    packer.pack_dram();
23    packer.pack_bram();
24    packer.pack_dsps();
25    packer.pack_ffs();
26    packer.finalise_muxfs();
27    packer.pack_lutffs();
28 }
```

Physical cell *templates* are built in `cells.cc`; for example every 6-LUT becomes a SLICE_LUTX with the full A1–A6 / WA1–WA8 port set (the WA ports exist so the same bel can serve as distributed RAM):

`himbaechel/uarch/xilinx/cells.cc:34`

```
1     IdString id = ctx->id(name);
2     cell->ports[id].name = id;
3     cell->ports[id].type = dir;
4 };
5 if (type == id_SLICE_LUTX) {
6     for (int i = 1; i <= 6; i++)
7         add_port("A" + std::to_string(i), PORT_IN);
8     for (int i = 1; i <= 9; i++)
9         add_port("WA" + std::to_string(i), PORT_IN);
10    add_port("DI1", PORT_IN);
11    add_port("DI2", PORT_IN);
12    add_port("CLK", PORT_IN);
13    add_port("WE", PORT_IN);
14    add_port("SIN", PORT_IN);
15    add_port("O5", PORT_OUT);
16    add_port("O6", PORT_OUT);
17    add_port("MC31", PORT_OUT);
18 } else if (type == id_SLICE_FFX) {
19     add_port("D", PORT_IN);
20     add_port("SR", PORT_IN);
21     add_port("CE", PORT_IN);
22     add_port("CLK", PORT_IN);
23     add_port("Q", PORT_OUT);
24 } else if (type == id_RAMD64E) {
25     for (int i = 0; i < 6; i++)
```

Primitive family	Packed to	Packer (uarch/xilinx)
LUT1-6, LUT6_2	SLICE_LUTX (O5/O6)	pack.cc (pack_luts)
FFs (FDRE/FDSE/...), latches	SLICE_FFX	pack.cc (pack_ffs)
MUXF7 / MUXF8	SELMUX2_1 trees	pack.cc (pack_muxfs)
CARRY4	MUXCY/XORCY reassembled chains	pack_carry.cc
SRL16E / SRLC32E	SLICE_LUTX (SRL mode)	pack.cc (pack_srls)
Distributed RAM (RAM64×1S/D, RAM32M/64M, 128/256×1S/D)	SLICEM LUTs + F7/F8 decode trees	pack_dram.cc
Block RAM (RAMB18E1 / RAMB36E1, TDP+SDP)	RAMB18E1/RAMB36E1 bels	pack.cc (pack_bram)
DSP48E1 (+ cascades)	DSP48E1_DSP48E1	pack_dsp_xc7.cc
IO buffers, IDDR/ODDR, ISERDESE2/OSERDESE2, IDELAYE2/ODELAYE2, IDELAYCTRL	IOB33 (HR) / IOB18 (HP) + IOLOGIC	pack_io.cc
BUFG/BUFGCE/BUFGCTRL, MMCME2, PLLE2, BUFR/BUFIO/BUFHCE	clocking sites	pack_clocking.cc
PS7 (Zynq)	PS7_PS7	pack_io.cc

Table 1: Primitive coverage of the xc7 uarch in this checkout (full matrix, with limitations, in the porting analysis).

7 Placement: shared algorithms, family legality

7.1 Dispatch

The generic Arch dispatches to one of three shared placers selected by `-placer`; the Xilinx uarch defaults to heap and maps its default router to `router2` (`xilinx.h:147`):

himbachel/arch.cc:266

```

1 bool Arch::place()
2 {
3     bool retVal = false;
4     uarch->prePlace();
5     std::string placer = str_or_default(settings, id("placer"), defaultPlacer);
6     if (placer == "heap") {
7         PlacerHeapCfg cfg(getCtx());
8         uarch->configurePlacerHeap(cfg);
9         cfg.ioBufTypes.insert(id("GENERIC_IOB"));
10        retVal = placer_heap(getCtx(), cfg);
11    } else if (placer == "static") {
12        PlacerStaticCfg cfg(getCtx());

```

```

13     uarch->configurePlacerStatic(cfg);
14     retVal = placer_static(getCtx(), cfg);
15 } else if (placer == "sa") {
16     retVal = placer1(getCtx(), Placer1Cfg(getCtx()));
17 } else {
18     log_error("Himbaechel architecture does not support placer '%s'\n",
19     placer.c_str());
20 }
21 uarch->postPlace();
22 getCtx()->settings[getCtx()->id("place")] = 1;
23 archInfoToAttributes();
24 return retVal;
25 }

```

7.2 HeAP: the analytical placer

placer_heap (“HeAP”) is the default. Each iteration:

1. **Bind constraints**—every BEL-attributed cell is placed with STRENGTH_USER.
2. **Solve**—cells are modelled as bound-to-bound springs; an Eigen Conjugate-Gradient solve produces x/y positions for each bel bucket run.
3. **Spread**—CutSpreader pushes overlapping cells apart per cell group, respecting group areas.
4. **Legalise**—StrictLegaliser walks cells to legal sites (a greedy random-walk with bel buckets and control-set pre-allocation), checking uarch legality on every candidate.
5. **Timing**—a timing pass if timing_driven.

The loop keeps the best snapshot, then finishes with a refinement pass (parallel_refine or placer1_refine). The alternatives are placer_static (ePlace-style electrostatic placement via FFT density solves with Nesterov acceleration—good for large designs) and placer1 (classic simulated annealing).

7.3 What the Xilinx uarch configures

The uarch teaches the placer about xc7 structure via configurePlacerHeap:

himbachel/uarch/xilinx/xilinx.cc:321

```

1 void XilinxImpl::configurePlacerHeap(PlacerHeapCfg &cfg)
2 {
3     cfg.hpwl_scale_x = 2;
4     cfg.hpwl_scale_y = 1;
5     cfg.beta = 0.5;
6     cfg.placeAllAtOnce = true;
7     cfg.get_cell_legalisation_weight = [this](Context *, CellInfo *ci) {
8         if (ci->type != id_SLICE_LUTX)
9             return 1;
10        auto tags = get_tags(ci);
11        // Place memory first, because they require entire SLICEMs
12        return tags->lut.is_memory ? 100 : 1;
13    };
14
15    cfg.ff_bel_bucket = id_SLICE_FFX;
16    cfg.ff_control_set_groups.resize(2);
17    for (int z = 0; z < 8; z++) {
18        cfg.ff_control_set_groups.at(z / 4).push_back((z << 4) | BEL_FF);
19        cfg.ff_control_set_groups.at(z / 4).push_back((z << 4) | BEL_FF2);
20    }
21    cfg.ctrl_set_max_radius = std::vector<int>{18, 15, 12, 9, 6, 3};
22
23    cfg.get_cell_control_set = [this](Context *, const CellInfo *ci) {

```

```

24         if (ci->type != id_SLICE_FFX)
25             return -1;
26         auto tags = get_tags(ci);
27         return tags->ff.control_set;
28     };
29 }

```

Two ideas matter here. **Clusters:** related physical cells (a CARRY4 and its LUTs, an F7MUX tree, a DRAM decode tree) are grouped as `ClusterIds`; the placer moves clusters atomically and asks the uarch for each member’s offset via `getClusterPlacement` (`arch.h:744–756`). **Control sets:** xc7 FFs in one half slice must share CLK/SR/CE (and their inversion bits), so the uarch pre-indexes control sets (`index_control_sets`, `xilinx.cc:617–635`) and hands the placer `ff_control_set_groups` plus a radius ladder (`ctrl_set_max_radius {18, 15, 12, 9, 6, 3}`) to keep same-control-set FFs near each other.

7.4 Legality: the heart of xc7 placement

The placer is family-agnostic; the uarch decides whether a set of cells may legally share a site. For logic tiles that is `XilinxImpl::xc7_logic_tile_valid(xilinx_place.cc:43)`, fed by per-tile `LogicTileStatus` caches that track which cells occupy which bel:

```

1  bool XilinxImpl::xc7_logic_tile_valid(IdString tile_type, const LogicTileStatus &lts)
2  {
3      bool is_slicem = (tile_type == id_CLBLM_L) || (tile_type == id_CLBLM_R);
4      bool tile_is_memory = false;
5      if (lts.cells[(3 << 4) | BEL_6LUT] != nullptr && get_tags(lts.cells[(3 << 4) |
6          BEL_6LUT])->lut.is_memory)
7          tile_is_memory = true;
8      bool small_memory = false;
9      if (lts.cells[(3 << 4) | BEL_5LUT] != nullptr && get_tags(lts.cells[(3 << 4) |
10         BEL_5LUT])->lut.is_memory)
11         small_memory = true;
12     NetInfo *wclk = nullptr;
13     // Check eight-tiles (mostly LUT-related validity)
14     for (int i = 0; i < 8; i++) {
15         if (lts.eights[i].dirty) {
16             lts.eights[i].dirty = false;
17             lts.eights[i].valid = false;
18
19             auto lut6 = get_tags(lts.cells[(i << 4) | BEL_6LUT]);
20             auto lut5 = get_tags(lts.cells[(i << 4) | BEL_5LUT]);
21
22             // Check 6LUT
23             if (lut6) {
24                 if (!is_slicem && (lut6->lut.is_memory || lut6->lut.is_srl)) {
25                     DBG();
26                     return false;
27                 } // Memory and SRLs only valid in SLICEMs
28                 if (lut6->lut.is_srl && (i >= 4)) {
29                     DBG();
30                     return false;
31                 }
32                 if (lut6->lut.is_memory || lut6->lut.is_srl) {
33                     if (wclk == nullptr)
34                         wclk = lut6->lut.wclk;
35                     else if (lut6->lut.wclk != wclk) {
36                         DBG();
37                         return false;
38                     }
39                 }
40             }
41         }
42     }
43     return tile_is_memory || small_memory;
44 }

```

```

37     }
38     if (lut5) {

```

The checker enforces, per eight (four LUT/FF columns) and per half slice:

- **SLICEM/SLICEL rules**—memory and SRL LUTs only in SLICEM (the “M” tiles), SRLs only in the bottom half;
- **5LUT coexistence**—a 5LUT may not sit under a 6-input or dual-output 6LUT; combined inputs must overlap sufficiently;
- **the X-input budget**—the F7MUX/F8MUX selects, the CARRY4 x-sigs, and any indirect FF D inputs must all agree on a single X net per position;
- **output-mux exclusivity**—O5, F7F8, carry-out and FF2 cannot all demand the single output mux;
- **half-tile control sets**—CLK/SR/CE plus their inversion, latch and sync bits must match across all eight FFs, and the distributed-RAM write clock must equal the FF clock in the bottom half.

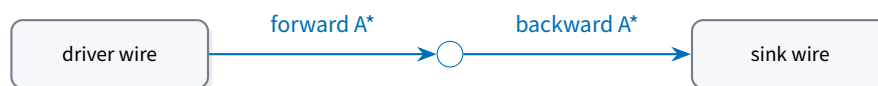
Known gap in this checkout

The analysis in `../xilinx-porting/03-algorithmic-diff.md` shows this checker still lacks several legality rules the openXC7 fork had to add over years of silicon debugging: the per-position site-exit (OUTMUX) budget, cross-position carry→FF pairing rejection, FF1-via-X + 5FF co-pack rejection, and SRL cascade placement. Porting those checks is work package WP1 of the porting plan—without them the legaliser can silently produce illegal slices.

After placement, `fixup_placement(xilinx_place.cc:389)` re-permutes LUT5/LUT6 inputs onto non-overlapping A pins and records the logical mapping in `X_ORIG_PORT_*` attributes—the bridge between nextpnr’s free LUT input ordering and RapidWright/Vivado’s fixed physical ordering.

8 Routing: router2 and the Xilinx glue

Routing uses the shared router2 (default) or router1. Router2 is a bidirectional meet-in-the-middle A* search:



meeting point found $\Rightarrow$ path = forward prefix + reverse of backward prefix

costs: delay + historical congestion + present congestion + resource-group value

Figure 5: Router2 searches from both ends of each arc and stitches the two halves at a meeting wire. Congestion is tracked per wire and per *resource group* (the uarch’s `getResourceKeyForPip()`), and the routing is checked by a router1 pass at the end.

The Xilinx uarch adds family-specific routing phases around the shared core (`preRoute/postRoute` in `xilinx.cc`):

- `find_source_sink_locs()`—precomputes the source and sink locations for each net for the router;
- `route_clocks()` (`pack_clocking.cc`)—routes the dedicated clock backbone (BUFG→HROW→CK_MUXED) before general routing so ordinary signals cannot claim clock resources;

- `fixup_routing()` (`xilinx_place.cc:512`)—after routing, converts any LUT-permutation pips the router used into physical connections and records the permutation in `X_ORIG_PORT_*` (again for RapidWright compatibility), and legalises OSERDESE3 bypasses.

8.1 Delay estimation

The router’s cost function needs wire and pip delays. Himbaechel computes them from the chipdb timing classes (see §9); the Xilinx uarch overrides `estimateDelay/predictDelay` (`xilinx.cc:735–780`) with heuristics that the source itself marks as improvable (TODO: improve sophistication here based on old `nextpnr-xilinx` code).

9 Timing

Timing data is baked into the chipdb as three POD families:

- **Pip timing**—per pip class: intrinsic delay, input capacitance, output resistance;
- **Node timing**—capacitance/resistance/delay for merged cross-tile nodes;
- **Cell timing**—per cell type: combinational arcs and sequential arcs (setup/hold/clock-to-Q).

At run time the generic Arch implements the shared timing-engine callbacks (`getCellDelay`, `getPortTimingClass`, `getPortClockingInfo`) by looking these PODs up (`arch.cc:470–520`); the Xilinx uarch only supplies the family-specific details—for instance BRAM timing variants (write-TDP vs. write-SDP $\times$ read-TDP vs. read-SDP) are selected by rewriting the cell’s timing index (`xilinx.cc:602–612`).

Clock constraints flow through two stages: `create_clock` periods from the XDC parser land on nets (`xdc.cc`), and `generate_constraints` (`pack_clocking.cc:384–535`) propagates them through BUFGCTRL/IBUF/PLL/MMCM, computing VCO and output-clock frequencies so downstream logic is analysed against derived clocks.

Timing limitations in this checkout

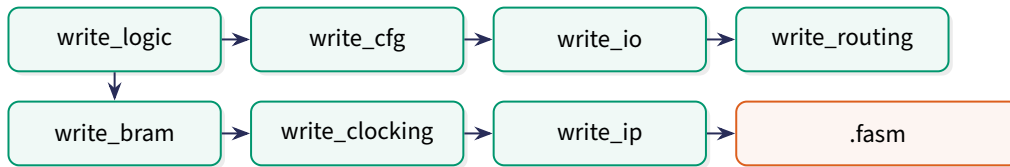
Only one speed grade (DEFAULT) is generated (`gen/xilinx_gen.py:387`), and LUT/FF timing is stubbed rather than imported from SDF (`gen/xilinx_gen.py:450–462`)—so timing-driven flows currently lean on BRAM/SELMUX2_1/CARRY4 arcs plus heuristics. This is a known limitation tracked in the porting analysis.

10 FASM: from routed design to bitstream

The final stage writes an *abstract bitstream* in FASM (FPGA Assembly) format—one human-readable line per configured feature:

FASM keeps the last step independent of Vivado: the features reference prjxray-documented bit positions, so the whole xc7 flow (Yosys $\rightarrow$ nextpnr $\rightarrow$ fasm2frames $\rightarrow$ xc7frames2bit) is fully open source. Pseudo-pips (config-only pips with no real connection) are handled by a `(tileType, dest, source)` table in the FASM backend, and LUT input permutations are re-encoded as `X_ORIG_PORT` attributes rather than emitted as pip bits.

11 Device support today



one FasmBackend (fasm.cc, 1864 lines): LUT/FF/carry config · IOLOGIC · BRAM widths/ints · BUFGCTRL/PLL/MMCM · DSP · pip bits

Figure 6: The FASM writer walks the routed design section by section and emits feature lines. External prjxray tools (fasm2frames, xc7frames2bit, via examples/bitgen_xray.sh) then assemble the physical bitstream.

Family	Site metadata (meta/)	Devices in default build
artix7	SLICEL/M, IOB33, GTPE2, PCIE_2_1, ...	xc7a50t, xc7a100t, xc7a200t (+ a35t alias)
kintex7	IOB18/33, GTXE2, ODELAYE2, ...	— (code path exists, not in default list)
spartan7	IOB33, ILOGICE3/OLOGICE3, ...	xc7s50
zynq7	+ PS7, IOB18/33	xc7z010, xc7z020

Table 2: Device coverage in this checkout. Kintex-7 metadata exists but no xc7k device is enabled by default—a one-line CMake change (see the porting plan WP0).

UltraScale(+) is *not* served by this uarch: `match_device` accepts only `xc7*` (xilinx.cc:815). The separate `ng-ultra` uarch targets a different UltraScale-class part (“Project Beyond”), and the E2/E3-era identifiers in the shared `constids.inc` are a superset vocabulary without packers. Porting the openXC7 fork’s UltraScale+ (xcup) flow is its own work package.

A Glossary

chipdb

The compile-time-generated, position-independent binary device database, memory-mapped at run time.

bel Basic Element of Logic—one atomic site resource (a LUT, a FF, a BRAM, a pad); “bel” in nextpnr.

pip Programmable Interconnect Point—one configurable switch between two wires (or between a site wire and a bel pin).

node

A merged group of electrically-connected tile wires; the unit of net identity across tile boundaries.

uarch

A himbaechel micro-architecture plugin (one `HimbaechelAPI` subclass) for a device family.

RelPtr / RelSlice

Relative pointer / relative slice—offset-based references that make the chipdb position-independent.

FASM

FPGA Assembly—the textual feature-per-line interchange format emitted by nextpnr and consumed by prjxray tools.

X_ORIG_PORT

Attribute recording the original logical LUT input ordering after nextpnr permutes physical inputs; keeps RapidWright/Vivado-compatible names.

B Where the details live

- Architecture analysis, feature inventory, algorithmic diff and the porting plan: `docs/xilinx-porting/` in this repository.
- Upstream framework documentation: `docs/himbaechel.md`.
- Chipdb POD definitions: `himbaechel/chipdb.h`.
- Xilinx uarch implementation: `himbaechel/uarch/xilinx/`.
- Shared placers/routers: `common/place/`, `common/route/`.